Bluetooth Master Mode

1. Bluetooth Master Overview

BLE (Bluetooth Low Energy) is a short-range wireless communication protocol designed for low power consumption and low bandwidth scenarios. Compared to classic Bluetooth, BLE connections are faster (millisecond-level) and consume less power (coin batteries can run for months).

Importance of BLE Master Tutorial: The BLE master (Central) plays the role of actively discovering and connecting to slaves, serving as the control end for data aggregation and command distribution. Phones, tablets, gateways, and other central devices all operate as masters—mastering master programming means being able to build central nodes that actively scan, auto-connect, and read/write data from remote slaves, which is the other core capability for building a complete BLE system.

This experiment uses the NimBLE protocol stack (saves ~50% RAM compared to Arduino's default Bluedroid) to scan for the target slave `"ESP-IDF"`, auto-connect and discover GATT services, subscribe to Notify to receive slave push data, and also write data to the slave every 5 seconds via a FreeRTOS task.

After completing this tutorial, you will have: An ESP32-S3 development board that automatically scans surrounding BLE devices upon power-on, auto-connects to a slave named `ESP-IDF`, subscribes to notifications, sends data every 5 seconds, and prints all received slave push data on the serial port—a complete bidirectional BLE communication master.

Typical Application Scenarios:

Application Type	Example
Mobile App	Scan and connect to bracelet, watch, and other sensor peripherals
Data Collection	Master collects data from multiple slave sensors and uploads to cloud
Door Lock Control	Phone (master) connects to door lock slave, sends unlock command

BLE Role System:

Role	GAP Name	Responsibility
Slave	Peripheral	Advertises its presence, accepts connections, provides GATT services for master access
Master	Central	Actively scans surrounding devices, initiates connections, discovers services and characteristics, reads/writes data

---

2. Core Concepts

2.1 GAP — Generic Access Profile

GAP (Generic Access Profile) defines how BLE devices **discover and connect** to each other.

GAP operations involved in this master:

Operation	Arduino API	Description
Scan	`NimBLEDevice::getScan()->start()`	Actively scan surrounding advertisements, collect device list
Connect	`pClient->connect(pDev)`	Initiate connection request to target slave
Disconnect Rescan	`ClientCallbacks::onDisconnect()` → `loop()` auto rescan	When disconnect is detected, `loop()` detects `connected=false` and automatically rescans and reconnects

2.2 Scan Modes

Mode	API	Behavior	Use Case
Active Scan	`setActiveScan(true)`	Send Scan Request after receiving advertisement to get Scan Response	Need complete device name or more advertisement data
Passive Scan	`setActiveScan(false)`	Silently collect advertisement packets only, send no requests	Low power listening, beacon collection

> This project uses active scan (`setActiveScan(true)`) to ensure obtaining the complete device name from the slave's Scan Response.

2.3 GATT — Generic Attribute Profile

GATT (Generic Attribute Profile) defines the data organization structure of BLE devices.

This project's custom GATT service architecture (defined on slave side, discovered by master)

```
Service: 0xFFF0 (Custom Primary Service)
  ├─ Characteristic: 0xFFF1
  |     Properties: READ | NOTIFY (Readable + Slave actively notifies master)
  |     Direction: Slave → Master
  |     Central Action: subscribe(true, callback) — Write CCCD to enable notification
  |     Purpose: Slave pushes data to master via Notify every 2 seconds
  |                     Slave pushes data to central via Notify every 2 seconds
  |
  └─ Characteristic: 0xFFF2
        Properties: WRITE | WRITE_NR (Supports both write with response and write without
response)
        Direction: Master → Slave
        Central Action: writeValue(data, false) — Write data to slave
        Purpose: Master sends data to slave every 5 seconds
                        Master sends data to slave every 5 seconds
```

2.4 Notify and Write

Operation	Initiator	Target	Ack Required	Arduino API	Typical Scenario
Notify	Slave → Master	0xFFF1	No	`subscribe(true, callback)` receive	Sensor data reporting, heartbeat count
Write	Master → Slave	0xFFF2	WRITE: Yes; WRITE_NR: No	`writeValue(data, false)` send	Control command distribution, parameter configuration

> In this project, the master writes `"Hello from ESP32 Master"` to 0xFFF2 every 5 seconds; the slave pushes data via 0xFFF1 Notify every 2 seconds, and the master receives and prints it immediately in `notifyCallback`. Both operations are asynchronous and independent, non-blocking each other.

2.5 CCCD — Client Characteristic Configuration Descriptor

CCCD (Client Characteristic Configuration Descriptor) UUID is fixed at `0x2902`. Notify is disabled by default—the master must write `0x0001` to the slave's CCCD to enable notifications.

CCCD Value	Meaning	Effect
`0x0000`	Disable notifications and indications	Slave does not push data
`0x0001`	Enable Notify	Slave starts pushing Notify to master
`0x0002`	Enable Indicate	Slave starts pushing Indicate to master (with acknowledgment)

```
// subscribe(true, callback) automatically completes two steps internally:
// 1. Write 0x0001 to slave CCCD (0x2902)
// 2. Register notifyCallback as the notification receive callback
pNotify->subscribe(true, notifyCallback);
```

> `subscribe(true, callback)` encapsulates the CCCD write process, so developers don't need to manually operate the descriptor. The second parameter is a callback function pointer for when Notify is received.

2.6 NimBLE-Arduino Library Architecture (Master Perspective)

ESP32-S3 uses **Apache NimBLE** as the BLE Host protocol stack. Master-side core class hierarchy:

```
NimBLEDevice                ← Static class, stack initialization + global control / Static class; stack init + global control
  ├─ NimBLEScan             ← Scan controller, manages scan params + results / Scan controller, manages scan params + results
  │    └─ NimBLEScanResults / NimBLEAdvertisedDevice  ← Scan results / Scan results
  ├─ NimBLEClient           ← GATT client, manages connection + service discovery / GATT client, manages connection + service discovery
  │    ├─ NimBLEClientCallbacks      ← User overrides onConnect/onDisconnect
  │    └─ NimBLERemoteService        ← Remote service reference / Remote service reference
  │         └─ NimBLERemoteCharacteristic  ← Remote characteristic reference, read/write/subscribe / Remote char ref, read/write/subscribe
  └─ (NimBLEServer / NimBLEAdvertising  ← Not used by master / Not used by central)
```

> Core differences between master and slave:

Component	Slave (Peripheral)	Master (Central)
Core Object	`NimBLEServer` — Provides GATT services	`NimBLEClient` — Discovers and accesses remote GATT services

Component	Slave (Peripheral)	Master (Central)
Characteristic Type	`NimBLECharacteristic` — Local characteristic	`NimBLERemoteCharacteristic` — Remote characteristic reference
Active Operations	`notify()` push / wait for writes	`subscribe()` subscribe / `writeValue()` write
Advertisement	`NimBLEAdvertising` send advertisements	`NimBLEScan` scan advertisements

---

3. Hardware Dependencies

3.1 ESP32-S3 BLE Hardware Specifications

Specification	Parameter
Wireless Standard	BLE 5.0
Max TX Power	+20 dBm
Receive Sensitivity	-97 dBm (1Mbps mode)
Max MTU	251 bytes (Auto-negotiated by NimBLE-Arduino)
Antenna	On-board PCB antenna
Scan Interval	Set to 10ms window in this tutorial
Protocol Stack	NimBLE (Apache open source) via NimBLE-Arduino
Arduino TX Power Setting	`NimBLEDevice::setPower(ESP_PWR_LVL_P9)` — Set to +9dBm (max) in this tutorial

3.2 Required Libraries

Library	Purpose	Include Method
`NimBLEDevice.h`	NimBLE BLE protocol stack Arduino wrapper	`#include <NimBLEDevice.h>`

> The `NimBLEDevice` library needs to be installed via Arduino IDE library manager: **Tools** → **Manage Libraries** → **Search "NimBLE"** → **Install "NimBLE-Arduino"** (author: h2zero). Do not confuse with ESP32's built-in "BLE" library (based on Bluedroid).

3.3 Companion Devices

You need a companion device with BLE slave program flashed onto ESP32-S3, or use a phone Bluetooth debugging assistant app to simulate a slave.

---

4. Project Architecture and Flow

File Structure (BLE_Master.ino, complete source ~150 lines):

```
BLE_Master.ino
├─ #include + Macro definitions (TARGET_DEVICE_NAME, SERVICE_UUID, CHAR_*_UUID)
├─ Global variables (pClient, pWriteChar, connected)
├─ void notifyCallback()        ← Receive slave Notify push (Section 5.2)
├─ class MyClientCallbacks      ← Connection/disconnection events (Section 5.3)
├─ void scanAndConnect()        ← Scan→match name→connect→discover service→subscribe
(Section 5.4)
├─ void sendTask()              ← FreeRTOS timed write (Section 5.5)
├─ void setup()                 ← BLE master initialization (Section 5.6)
└─ void loop()                  ← Auto scan reconnect when disconnected (Section 5.7)
```

Program Flow:

```
setup()
  ├─ NimBLEDevice::init("MASTER")          // Initialize stack
  └─ xTaskCreate(sendTask)                 // Create FreeRTOS timed send task

loop()
  └─ if (!connected) → scanAndConnect()    // Continuously scan and reconnect when
disconnected

scanAndConnect():
  ├─ Scan 500ms, iterate results
  ├─ Match device name "ESP-IDF"
  |    └─ When name is empty: manually parse AD Structure to extract device name
  ├─ createClient() + connect()            // Create client and initiate connection
  ├─ getService(0xFFF0)                    // Get target service
  |    ├─ getCharacteristic(0xFFF1) → subscribe(notifyCallback)
  |    └─ getCharacteristic(0xFFF2) → Save pWriteChar reference

sendTask (FreeRTOS):
  └─ Every 5s: writeValue("Hello from ESP32 Master", false)  // WRITE_NR

notifyCallback():
  └─ Print slave push data

MyClientCallbacks:
  ├─ onConnect()    → connected = true, print log
  └─ onDisconnect() → connected = false, pWriteChar = nullptr, print log
```

---

5. Source Code Details

Source code: `Source code → Arduino → 3.Network Communication → BLE_Master → BLE_Master.ino`

5.1 Header Files and Global Variables

The top of the file contains `#include`, macro definitions, and global variables. `NimBLEDevice.h` is the only header file, which internally aggregates all C++ wrapper classes of the NimBLE protocol stack. The UUID values in macros must exactly match the slave side. Three global variables—client pointer, remote Write characteristic reference, and connection status flag—persist throughout the program's lifecycle. `pWriteChar` is set to `nullptr` when disconnected to prevent wild pointer access to invalid remote characteristic references.

```
// NimBLE protocol stack / NimBLE BLE stack
#include <NimBLEDevice.h>

// Target slave advertising name / Target slave advertising name
#define TARGET_DEVICE_NAME   "ESP-IDF"
// Primary service UUID / Primary service UUID
#define SERVICE_UUID         NimBLEUUID((uint16_t)0xFFF0)
// Notify char — slave→central / Notify char — slave→central
#define CHAR_NOTIFY_UUID     NimBLEUUID((uint16_t)0xFFF1)
// Write char — central→slave / Write char — central→slave
#define CHAR_WRITE_UUID      NimBLEUUID((uint16_t)0xFFF2)

// BLE GATT client pointer / BLE GATT client pointer
NimBLEClient* pClient = nullptr;
// Reference to slave's write characteristic / Reference to slave's write characteristic
NimBLERemoteCharacteristic* pWriteChar = nullptr;
// BLE connection status flag / BLE connection status flag
bool connected = false;
```

5.2 Receiving Slave Push — notifyCallback

`notifyCallback` is the callback function for the master to receive slave Notify data. Each time the slave calls `notify()` to push data, the NimBLE protocol stack automatically calls this function. `data` points to the raw byte array, `len` is the byte count, and `isNotify` is `true` for Notify (`false` for Indicate). `Serial.write(data, len)` directly outputs raw bytes without string conversion to avoid `\0` truncation issues.

```
// Slave notify data callback / Slave notify data callback
void notifyCallback(NimBLERemoteCharacteristic* pChar, uint8_t* data, size_t len, bool
isNotify) {
  // Print label / Print label
  Serial.print("Slave:");
  // Output raw bytes / Output raw bytes
  Serial.write(data, len);
  Serial.println();
}
```

5.3 Connection/Disconnection Event Callbacks — MyClientCallbacks

Connection and disconnection are two key events in the BLE master lifecycle. Inherit `NimBLEClientCallbacks` and override `onConnect` and `onDisconnect`: set the status flag when connected for the send task to use; when disconnected, besides clearing the flag, **must set** `pWriteChar` **to** `nullptr`—remote characteristic references are invalid after connection disconnect, and the next reconnection requires re-obtaining through

GATT discovery, otherwise write operations will access invalid pointers.

```
// BLE connection callbacks class / BLE connection callbacks class
class MyClientCallbacks : public NimBLEClientCallbacks {
  void onConnect(NimBLEClient* pClient) {
    // Set connection flag / Set connection flag
    connected = true;
    Serial.println("Connected successfully!");
  }

  void onDisconnect(NimBLEClient* pClient, int reason) {
    // Clear connection flag / Clear connection flag
    connected = false;
    // Key: nullify stale remote reference after disconnect / Key: nullify stale remote
reference after disconnect
    pWriteChar = nullptr;
    Serial.println("Disconnect and rescan...");
  }
} clientCB;  // Global instance / Global instance
```

5.4 Scanning and Connecting — scanAndConnect

`scanAndConnect` is the master's most core function, completing the entire process of scan → match → connect → service discovery → subscribe.

Scan parameter configuration: Active scan + 10ms window + 500ms duration + don't filter duplicate advertisements, to discover all devices in a short time as much as possible.

Device name extraction: `pDev->getName()` sometimes returns an empty string (the device name in the slave advertisement packet may be in the Scan Response rather than the advertisement packet body), the code manually parses the Type `0x09` (Complete Local Name) or `0x08` (Shortened Local Name) field in the AD Structure to extract the name. This is a common pitfall in BLE master development—**do not rely solely on `getName()` to determine the device**.

Post-connection GATT discovery: After finding the target slave, four steps are performed in sequence: create Client → connect → get Service → get two Characteristics. Call `subscribe(true, notifyCallback)` on the Notify characteristic to enable notifications; save `pWriteChar` reference for the Write characteristic for the send task to use.

```
void scanAndConnect() {
  Serial.println("\nScan BLE devices...");

  // Get scanner / Get scanner
  NimBLEScan* pScan = NimBLEDevice::getScan();
  // Active scan / Active scan
  pScan->setActiveScan(true);
  // Scan interval 10ms (16 × 0.625ms) / Scan interval 10ms
  pScan->setInterval(16);
  // Scan window 10ms / Scan window 10ms
  pScan->setWindow(16);
  // Don't filter duplicates / Don't filter duplicates
  pScan->setDuplicateFilter(false);
```

```
  // Scan 500ms, non-blocking / Scan 500ms, non-blocking
  pScan->start(500, false);
  // Wait for scan to finish / Wait for scan to finish
  delay(500);

  // Get scan results / Get scan results
  NimBLEScanResults results = pScan->getResults();
  int count = results.getCount();

  Serial.print("Number of discovered devices: ");
  Serial.println(count);

  // Iterate all discovered devices / Iterate all discovered devices
  for (int i = 0; i < count; i++) {
    const NimBLEAdvertisedDevice* pDev = results.getDevice(i);
    String name = pDev->getName().c_str();
    // BD_ADDR (6-byte Bluetooth device address) / BD_ADDR (6-byte Bluetooth device address)
    String mac  = pDev->getAddress().toString().c_str();

    // ============================================
    // Key: getName() may be empty; must parse AD Structure manually
    // Key: getName() may be empty; must parse AD Structure manually
    // ============================================
    if (name.isEmpty()) {
      // Get raw advertising payload bytes / Get raw advertising payload bytes
      std::vector<uint8_t> raw = pDev->getPayload();
      String payload = String((const char*)raw.data(), raw.size());
      size_t pos = 0;
      while (pos + 2 < payload.length()) {
        // Field total length (includes self) / Field total length (includes self)
        uint8_t fieldLen  = (uint8_t)payload[pos];
        // AD Type identifier / AD Type identifier
        uint8_t fieldType = (uint8_t)payload[pos + 1];
        // length=0 means end of advertising data / length=0 means end of advertising data
        if (fieldLen == 0) break;
        // AD Type 0x09 = Complete Local Name, 0x08 = Shortened Local Name
        if (fieldType == 0x09 || fieldType == 0x08) {
          // Extract name / Extract name
          name = payload.substring(pos + 2, pos + 2 + fieldLen - 1);
          break;
        }
        // Skip to next AD field / Skip to next AD field
        pos += fieldLen + 1;
      }
    }

    // Print discovered device info / Print discovered device info
    Serial.print("Equipment: ");
    if (name.isEmpty()) {
      Serial.print("(unknown) ");
      Serial.print(mac);
    } else {
      Serial.print(name);
```

```
      Serial.print(" [");
      Serial.print(mac);
      Serial.print("]");
    }
    Serial.println();

    // Match the preset target device name / Match the preset target device name
    if (name == TARGET_DEVICE_NAME) {
      Serial.println("Target slave device found!");
      // Stop scanning once target found / Stop scanning once target found
      pScan->stop();

      // Create BLE GATT client / Create BLE GATT client
      pClient = NimBLEDevice::createClient();
      // Register connection status callbacks / Register connection status callbacks
      pClient->setClientCallbacks(&clientCB);

      // Initiate BLE connection / Initiate BLE connection
      if (pClient->connect(pDev)) {
        // Get the target service / Get the target service
        NimBLERemoteService* pSvc = pClient->getService(SERVICE_UUID);
        if (pSvc) {
          Serial.println("Service 0xFFF0 found");

          // Get notify characteristic and subscribe / Get notify characteristic and
subscribe
          NimBLERemoteCharacteristic* pNotify = pSvc->getCharacteristic(CHAR_NOTIFY_UUID);
          if (pNotify) {
            // Write CCCD to enable Notify / Write CCCD to enable Notify
            pNotify->subscribe(true, notifyCallback);
            Serial.println("Notification subscription succeeded");
          }

          // Get write characteristic / Get write characteristic
          pWriteChar = pSvc->getCharacteristic(CHAR_WRITE_UUID);
          if (pWriteChar) Serial.println("Writable features found");
        }
      }
      return;
    }
  }
}
```

5.5 FreeRTOS Timed Write — sendTask

`sendTask` is an independent FreeRTOS task that checks connection status every 5 seconds in an infinite loop, and if online, writes data to the slave via the Write characteristic (0xFFF2). The second parameter `false` in `writeValue(data, false)` means using WRITE_NR (Write Without Response) mode—no slave acknowledgment required, suitable for periodic data distribution. If you need to ensure the slave receives it, change the second parameter to `true` (Write With Response, requires link layer acknowledgment).

```
// FreeRTOS send task / FreeRTOS send task
```

```
void sendTask(void* param) {
  while (1) {
    // Send only when connected and char is valid / Send only when connected and char is
valid
    if (connected && pWriteChar) {
      const char* sendMsg = "Hello from ESP32 Master";
      // Write Without Response / Write Without Response
      pWriteChar->writeValue(sendMsg, false);
      Serial.printf("Sent:%s\n", sendMsg);
    }
    // Block 5 seconds / Block 5 seconds
    vTaskDelay(pdMS_TO_TICKS(5000));
  }
}
```

5.6 setup() — Initialize BLE Master

The master's `setup()` is more concise than the slave's—no need to create GATT services, only need to initialize the stack and create the send task. The name in `NimBLEDevice::init("MASTER")` is the master's own identifier (not mandatory, not exposed during scan phase). TX power is set to +9dBm (`ESP_PWR_LVL_P9`), the master should use max power to ensure scan and connection range covers the slave.

```
void setup() {
  // Init serial + wait for USB enumeration / Init serial + wait for USB enumeration
  Serial.begin(115200);
  delay(2000);
  Serial.println("=========================");
  Serial.println("      BLE Host Start");
  Serial.println("=========================");

  // Init BLE stack / Init BLE stack
  NimBLEDevice::init("MASTER");
  // TX power +9dBm (max) / TX power +9dBm (max)
  NimBLEDevice::setPower(ESP_PWR_LVL_P9);

  // Create FreeRTOS send task / Create FreeRTOS send task
  xTaskCreate(sendTask, "send", 4096, NULL, 1, NULL);
}
```

BLE Master and Slave Initialization Comparison

Step	Slave (Peripheral)	Master (Central)
1. Initialize stack	`NimBLEDevice::init(DEVICE_NAME)`	`NimBLEDevice::init("MASTER")`
2. Create server/client	`createServer()` + `setCallbacks()`	`createClient()` + `setClientCallbacks()` (in `scanAndConnect`)
3. Create service	`createService()` + `createCharacteristic()`	**No need to create** — Discover slave services via GATT discovery
4. Start advertising/scanning	`startAdvertising()`	`getScan()->start()` (in `scanAndConnect`)

Step	Slave (Peripheral)	Master (Central)
5. Create task	`xTaskCreate(sendTask)`	`xTaskCreate(sendTask)`

> Core difference: Slave builds local GATT service table (`createService` + `createCharacteristic`) for master to discover; Master discovers and references slave's remote services via `getService` + `getCharacteristic`.

5.7 loop() — Auto Reconnect Loop

`loop()` executes in Arduino's main task in a loop, checking connection status once every 1 second. When not connected, it calls `scanAndConnect()` to initiate scanning—this is a one-time scan+connection flow; if the slave is found and connected, `connected` is set to `true`, and `loop()` enters an empty loop waiting; if the scan doesn't find the target or connection fails, it rescans after `delay(1000)`. This forms a robust "auto reconnect on disconnect" mechanism—no reset needed to recover from any state.

```
void loop() {
  // If disconnected: scan and connect / If disconnected: scan and connect
  if (!connected) {
    scanAndConnect();
  }
  // Check again after 1 second / Check again after 1 second
  delay(1000);
}
```

Execution Flow Distribution

Execution Flow	Responsible For	Runs On
NimBLE Host task	Protocol stack event loop, handles scan results, connection callbacks, GATT discovery, Notify callbacks	Auto-created inside `NimBLEDevice::init()`
`sendTask`	Sends data to slave via Write every 5 seconds	User `xTaskCreate` created
`loop()`	Continuously monitors connection status, triggers `scanAndConnect()` to reconnect on disconnect	Arduino main task

6. Operation Flow

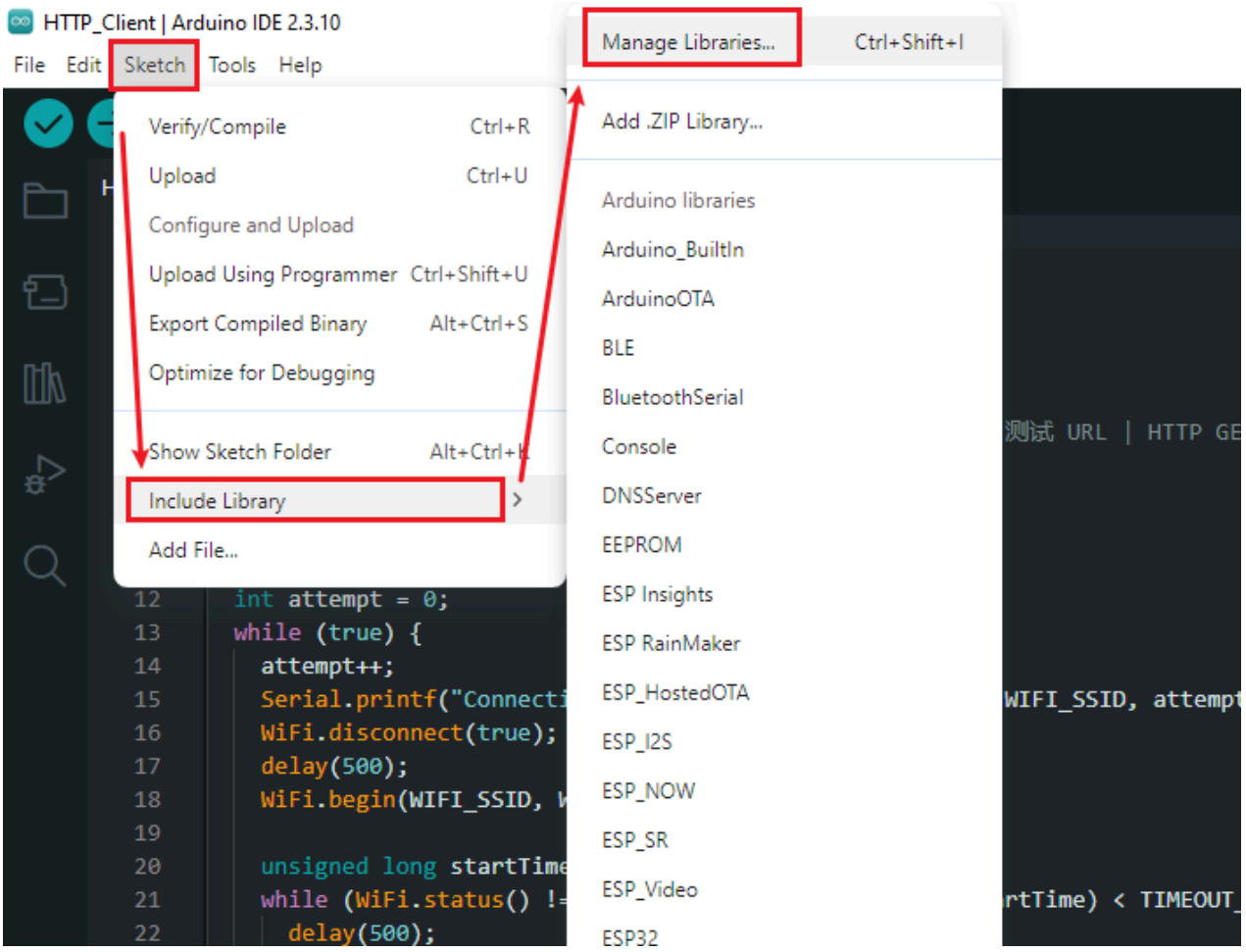
> There are two ways to test the master, choose one: **Method 1 (Dual-board communication)** requires two ESP32-S3 boards with Master and Slave sketches flashed respectively; **Method 2 (Phone simulated slave)** is suitable when you only have one development board.

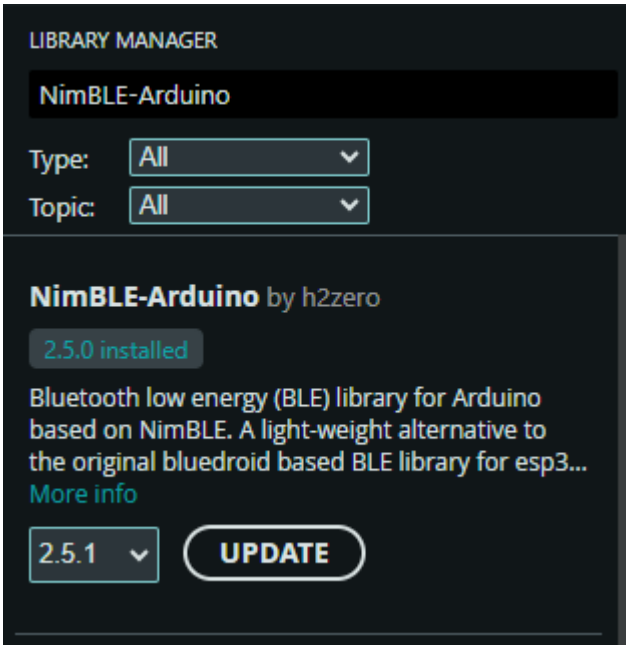
6.1 Build and Flash

Prerequisites:

1. **NimBLE-Arduino library installed (Tools → Manage Libraries → Search "NimBLE" → Install, author h2zero)**.

   Slave device ready — Another ESP32-S3 flashed with BLE Slave program, or prepare to use a phone app as the slave.





Arduino flashing flow: See `Arduino → 1. Before Development → 2. Build and Flash`.

6.2 Method 1: Dual-Board Communication Test (Requires Two Boards)

Applicable scenario: Two ESP32-S3 boards, one as BLE Master, one as BLE Slave, implementing complete two-board communication.

Step	Operation	Expected Result
1	Slave board powered on	Slave serial outputs `Waiting for host connection...`
2	Master board powered on	Master scans for `ESP-IDF` and auto-connects
3	Wait for connection	Master serial outputs `Connected successfully!`
4	Wait ~2 seconds	Master serial prints `Slave: Hello from ESP32 Slave`
5	Wait ~5 seconds	Master serial prints `Sent: Hello from ESP32 Master`
6	Disconnect Slave power	Master serial outputs `Disconnect and rescan...`, automatically rescans
7	Slave re-powered on	Master rediscovering and reconnecting, communication resumes

6.3 Method 2: Phone Simulated Slave Test

Applicable scenario: Only one ESP32-S3, using a phone app as the BLE slave, verifying the master's scan, connection, and write functions.

Scanning for slave devices:

```
Scan BLE devices...
Number of discovered devices: 50
Equipment: (unknown) 71:b5:fa:11:63:74
Equipment: (unknown) 18:d1:3f:df:3c:be
Equipment: (unknown) c0:8a:a3:fb:2f:1d
Equipment: (unknown) 6e:85:fc:b9:f1:e2
Equipment: (unknown) 11:01:4e:95:a5:76
Equipment: ESP-IDF [a4:cb:8f:c6:b8:65]
Equipment: (unknown) 21:7b:f0:6b:3e:f8
Equipment: (unknown) 1f:66:cd:88:07:eb
Equipment: (unknown) 40:00:07:1b:5f:b2
Equipment: (unknown) 56:76:2d:e5:93:23
Equipment: Baseus BowieE3 [41:aa:67:1b:51:ab]
Equipment: (unknown) 40:f6:79:02:ce:e6
Equipment: (unknown) e0:8c:fe:42:d7:7a
Equipment: (unknown) 53:33:24:47:c3:07
Equipment: (unknown) 46:04:31:ec:e9:29
Equipment: (unknown) 77:82:07:5a:42:77
Equipment: (unknown) 5e:60:ba:96:27:04
Equipment: (unknown) 5c:14:ca:64:da:71
Equipment: (unknown) 7f:99:94:87:f2:45
Equipment: (unknown) 45:ef:7c:00:57:40
Equipment: (unknown) 54:dc:cb:14:5a:55
Equipment: (unknown) 6c:0d:c4:25:0f:7c
Equipment: midea [d8:34:1c:c2:48:fb]
Equipment: (unknown) 33:21:e8:67:0d:5e
Equipment: (unknown) 76:a6:6e:60:b7:a2
Equipment: (unknown) d4:81:7f:ab:60:02
Equipment: NZop5Rb8xxmcGFt1XgDn8ej58 [53:04:8e:15:7c:e1]
Equipment: (unknown) 0b:d8:1a:48:38:ce
Equipment: (unknown) 6d:93:9a:64:af:9b
Equipment: (unknown) 49:42:3f:ba:9f:7d
Equipment: (unknown) 72:b8:75:34:a3:14
Equipment: (unknown) 40:00:05:8f:c0:2d
Equipment: (unknown) 5c:39:7a:98:eb:e6
Equipment: ESP [40:ca:4a:11:e9:d4]
```

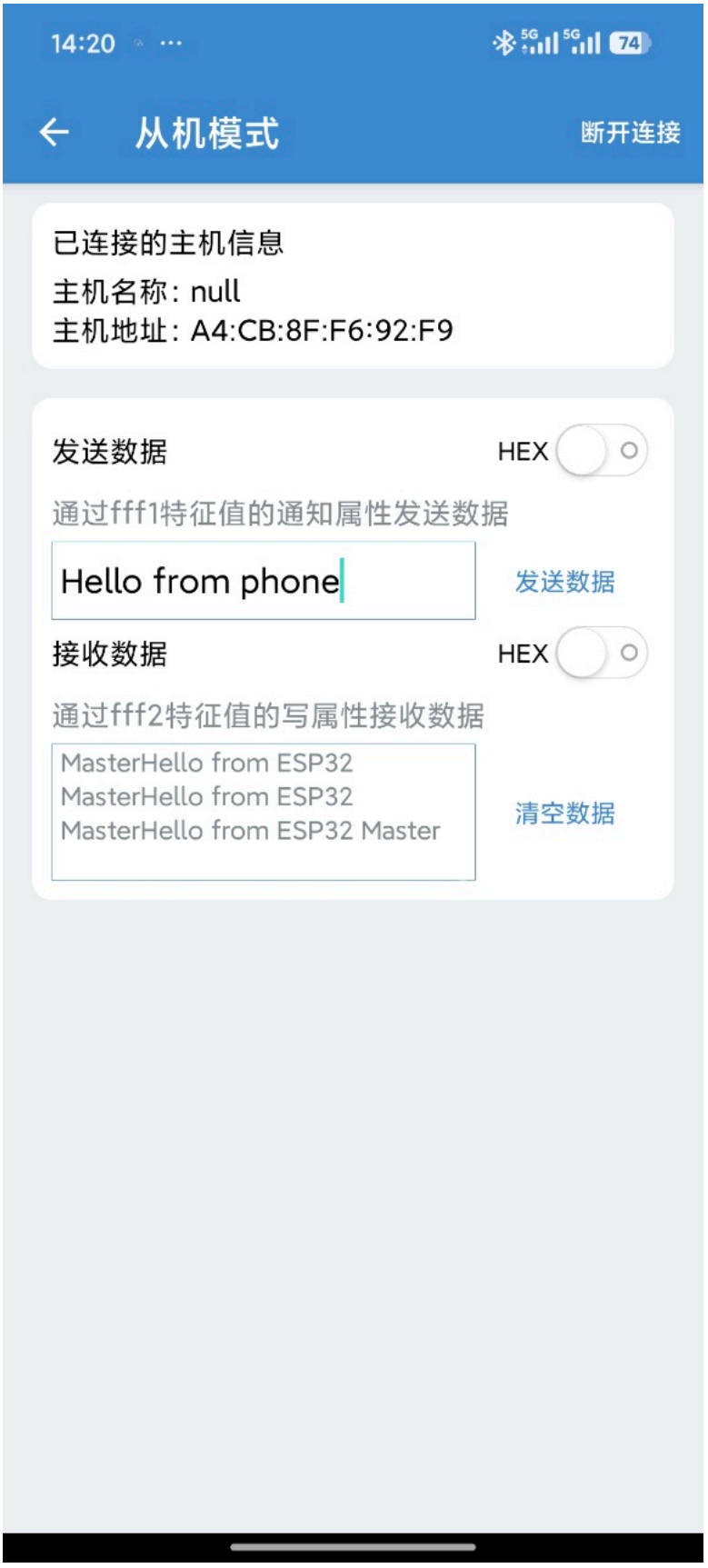
Connection successful

```
Target slave device found!
Connected successfully!
Service 0xFFF0 found
Notification subscription succeeded
Writable features found
```

Phone settings:

①: Switch to device mode

②Modify your Bluetooth ID and service numbers



③Serial monitor actual interaction:

Recommended Apps: **nRF Connect for Mobile** (most feature-rich), LightBlue, EFR Connect, BLE Scanner, all free.

nRF Connect's built-in **Advertising** function can simulate BLE slave:

Operation Steps:

Step	Operation	Expected Result
1	Open phone Bluetooth debugging tool → Switch to `ADVERTISER` tab	Enter advertisement configuration interface
2	Click `+` to create new advertisement → Set device name to `ESP-IDF`	Advertisement configuration ready
3	Add Service UUID `0xFFF0` → Turn on advertisement switch	Phone starts simulating slave advertisement
4	Master board powered on	Scans for phone-simulated `ESP-IDF` and auto-connects
5	Master writes to 0xFFF2 every 5s	Can see `Hello from ESP32 Master` on nRF Connect
6	Turn off phone advertisement	Master serial outputs `Disconnect and rescan...`

> **Note**: The phone only simulates advertisement+connection, may not actively push Notify data (depends on the app's GATT Server functionality), so Master-side `Slave:` log may not appear. This method mainly verifies the master's scan connection and write functions.

6.4 Expected Output — Dual-Board Communication

Master serial:

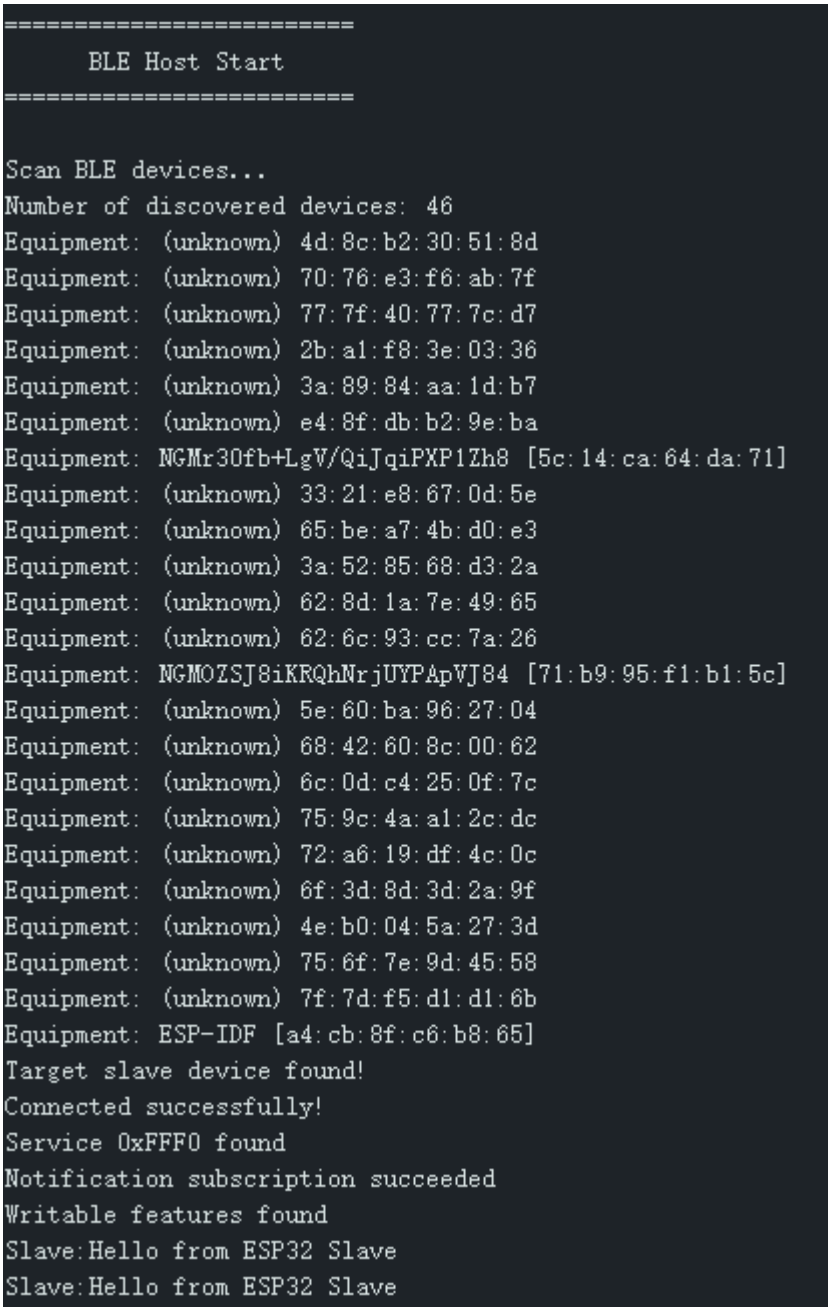
```
========================
      BLE Host Start
========================

Scan BLE devices...
Number of discovered devices: 3
Equipment: ESP-IDF [AA:BB:CC:DD:EE:FF]
Target slave device found!
Connected successfully!
```

```
Service 0xFFF0 found
Notification subscription succeeded
Writable features found
Slave:Hello from ESP32 Slave
Sent:Hello from ESP32 Master
Slave:Hello from ESP32 Slave
Slave:Hello from ESP32 Slave
Sent:Hello from ESP32 Master
...
Disconnect and rescan...

Scan BLE devices...
```

Power-on, scanning, connecting:



Interaction with slave:

> **Output rhythm difference**: Slave `sendTask` pushes Notify at fixed 2-second interval, serial prints one `Sent:` every 2 seconds. Master `sendTask` writes at fixed 5-second interval. The different print frequencies on both ends are determined by their respective task cycles and don't affect BLE transmission—every notification and write is delivered at the NimBLE underlying layer.

6.5 Log Field Description

Log	Meaning
`BLE Host Start`	Program started, initialization complete
`Scan BLE devices...`	Start scanning surrounding BLE devices
`Number of discovered devices: N`	Number of devices discovered during scan
`Equipment: ESP-IDF [MAC]`	Scanned device name and Bluetooth MAC address
`Target slave device found!`	Target slave matched, stop scanning
`Connected successfully!`	BLE connection established successfully
`Service 0xFFF0 found`	GATT service discovery successful, found slave's primary service
`Notification subscription succeeded`	CCCD write successful, subscribed to slave Notify
`Writable features found`	Write characteristic reference obtained successfully, ready to send data
`Slave: <content>`	Received Notify data pushed by slave via 0xFFF1
`Sent: <content>`	Master wrote data to slave via 0xFFF2
`Disconnect and rescan...`	Connection disconnected, about to rescan

6.6 Edge Cases

Scenario	Behavior
Master powered on first (no slave online)	Attempts to scan 500ms every 1 second, repeatedly prints `Scan BLE devices...` until slave appears
Slave loses power (master online)	`onDisconnect` triggered → `connected=false`, `pWriteChar=nullptr` → `loop()` detects disconnect, rescans
Slave powered on again	Master matches `ESP-IDF` in next scan cycle, auto reconnects, re-discovers services, re-subscribes
Multiple devices with same name `ESP-IDF` in area during scan	Connects to first matched device in scan list (sorted by RSSI, strongest signal first)
`getName()` returns empty string	Manually parse AD Type 0x09/0x08 from raw advertisement payload to extract device name
Master resets after disconnect	`pWriteChar` set to `nullptr` in `onDisconnect`, next reconnect `scanAndConnect` re-obtains `getCharacteristic` for new reference
`sendTask` executes during disconnect	`connected=false` and `pWriteChar=nullptr`, condition `connected && pWriteChar` is false, skip this send

7. FAQ

Symptom	Cause	Solution
Compilation error `NimBLEDevice.h` not found	NimBLE-Arduino library not installed, or wrong BLE library installed (Bluedroid)	Library manager search "NimBLE-Arduino" (author h2zero) to install, confirm board selected `ESP32S3 Dev Module`, restart IDE
No serial output	USB CDC On Boot not enabled, or baud rate mismatch	Tools → USB CDC On Boot → Enabled; serial monitor set to 115200; press RST to reset
Cannot scan for slave	Slave not running, device name mismatch, too far or signal interference	Confirm slave program flashed and powered on, `DEVICE_NAME` = `"ESP-IDF"` matches; move within 5 meters; away from 2.4GHz interference sources
Device scanned but shows `(unknown)`	Slave advertisement packet doesn't contain complete device name (name in Scan Response)	**Normal** — Code already handles AD Structure manual parsing, doesn't affect connection; if still not matching, confirm slave side `setName()` was called

Symptom	Cause	Solution
`connect()` returns false	Too far, signal interference, or slave already connected by another device	Move closer to slave; reduce metal obstacles; confirm slave not connected by phone or other master (only one connection allowed at a time)
Cannot receive Notify after `subscribe`	Slave characteristic doesn't have NOTIFY property, CCCD write failed	Confirm slave 0xFFF1 characteristic declares `NIMBLE_PROPERTY::NOTIFY`; confirm slave `vNotifyTask` started
`writeValue()` write failed	Slave characteristic doesn't have WRITE property	Confirm slave 0xFFF2 characteristic declares `NIMBLE_PROPERTY::WRITE` or `WRITE_NR`
Disconnects immediately after connection	Incompatible connection parameters, slave-side `onConnect` timeout	Normal — `loop()` automatically rescans and reconnects; check distance and interference if persistent
`getName()` still empty after connection	`getPayload()` returns raw advertisement packet data, AD Structure parsing may have errors	Check if slave side simultaneously calls `addServiceUUID()` and `setName()`; use nRF Connect packet capture to confirm advertisement data
Slave/Master output frequency inconsistent	Both ends' prints driven by their respective task cycles, Slave 2s, Master 5s	Normal — doesn't affect BLE transmission, every notification and write delivered at NimBLE underlying layer
Can Wi-Fi and BLE be used simultaneously?	ESP32-S3 supports time-division multiplexing, Arduino core and NimBLE auto-coordinate	Include both `#include <WiFi.h>` and `<NimBLEDevice.h>`; note throughput decreases for both, audio streaming not recommended coexisting
Can Master connect to multiple slaves simultaneously?	NimBLE supports multiple connections, but requires additional management of multiple Client instances	Create multiple `NimBLEClient` objects, each with independent `connect()` + `getService()`; note memory overhead and scheduling